What is Function Caching?

Save the result of a function when it's called with a certain input, and if it's called again with the same input, return the saved result instead of calculating again.

- Save time
- Improve performance
- Avoid repeating heavy calculations

Real-Life Analogy

Imagine you ask your friend:
"What is 5 squared?" — He calculates and says **25**.

Now, if you ask him again:
"What is 5 squared?" — He says:
"I already told you — it's 25!" — No need to calculate again.

How to Do Function Caching in Python?

Python provides a **built-in decorator** called `lru_cache` from the `functools` module.

```
from functools import lru_cache
import time

@lru_cache(maxsize=None)  # No limit on number of cached calls
def square(n):
    time.sleep(4)
    print(f"Calculating square of {n}")
    return n * n
```

Example Usage

```
print(square(5))  # First time — calculates and prints
print(square(5))  # Second time — uses cache, no calculation
```

Output:

```
Calculating square of 5
25
25
```

Notice: "Calculating…" only prints once — because second time it's using **cached result**.

What is a Regular Expression?

A **Regular Expression** (or **RegEx**) is a pattern used to **search, match, or replace** strings.

Think of it as a **smart search tool** that can find patterns — like phone numbers, emails, dates, or specific words in a string.

Python's RegEx Module: `re`

To use regular expressions in Python, you must import the built-in `re` module:

```
import re
```

Basic Functions in `re` Module

Function	Description
`re.search()`	Search for the pattern in the string
`re.match()`	Match pattern **only at the beginning**
`re.findall()`	Find **all matches** and return a list
`re.sub()`	Replace matched patterns with something
`re.compile()`	Compile a pattern for reuse

Example 1: `re.search()`

```
import re

text = "My phone number is 123-456-7890"
result = re.search(r'\d{3}-\d{3}-\d{4}', text)

print(result.group())  # Output: 123-456-7890
```

`\d` means digit, `{3}` means exactly 3 times

RegEx Patterns (Cheat Sheet)

Symbol	Meaning	Example	Matches
`.`	Any character	`a.c`	`abc`, `a1c`
`\d`	Any digit (0-9)	`\d\d`	`12`, `45`
`\D`	Non-digit	`\D+`	`abc`, `-+`
`\w`	Word character (a-z, A-Z, 0-9, _)	`\w+`	`hello`, `abc123`
`\s`	Whitespace	`\s+`	space, tab, newline
`^`	Starts with	`^Hi`	`Hi there`
`$`	Ends with	`end$`	`The end`
`[]`	Set of characters	`[aeiou]`	`a`, `e`, `i`...
`` ` ``	`` ` ``	OR operator	`` `cat

Symbol	Meaning	Example	Matches
*	0 or more	lo*	l, lo, loo...
+	1 or more	lo+	lo, loo
{n}	Exactly n times	\d{3}	123, 456

Example 2: `re.findall()`

```
text = "My numbers are 123 and 456"
matches = re.findall(r'\d+', text)
print(matches)  # ['123', '456']
```

Example 3: re.sub() for Replace

```
text = "My email is test123@gmail.com"
updated = re.sub(r'\w+@\w+\.\w+', 'hidden@email.com', text)
print(updated)
```

Example 4: re.match() vs re.search()

```
re.match(r'Hello', 'Hello World')   # Match at beginning
re.search(r'World', 'Hello World')  # Found anywhere
```

Using re.compile() (for reuse)

```
pattern = re.compile(r'\d+')
print(pattern.findall("I have 2 apples and 10 bananas"))
```

Use Cases of RegEx

- Validating email or phone number
- Extracting data from text or HTML
- Replacing sensitive info (e.g., mask Aadhaar)
- Log file analysis, scraping, etc.

Quick Email Matching Example

```
email = "My email is neeraj.kath@gmail.com"
match = re.search(r'[a-zA-Z0-9._]+@[a-z]+\.[a-z]+', email)
print(match.group())  # neeraj.kath@gmail.com
```

What is `asyncio`?

`asyncio` is a **Python module for writing asynchronous programs** — especially useful when you're doing I/O-bound tasks like:

- Reading/writing files
- Making API calls
- Waiting for a timer
- Network/socket programming

Instead of **waiting** for a task to finish, it **lets other tasks run** — increasing efficiency.

Real-Life Analogy

Imagine you're cooking 3 dishes:

- One needs boiling water
- One needs frying
- One needs baking

Instead of waiting for each step to finish, you:

- Start boiling → switch to frying → switch to baking → and rotate between them.

That's **asynchronous multitasking**.

Traditional (Synchronous) vs Async

Synchronous Code:

```
import time

def task1():
    time.sleep(2)
    print("Task 1 done")

def task2():
    time.sleep(2)
    print("Task 2 done")

task1()
task2()
```

Total time: 4 seconds

Asynchronous Code using `asyncio`:

```
import asyncio

async def task1():
    await asyncio.sleep(2)
    print("Task 1 done")

async def task2():
```

```
    await asyncio.sleep(2)
    print("Task 2 done")

async def main():
    await asyncio.gather(task1(), task2())

asyncio.run(main())
```

Total time: 2 seconds (both tasks sleep *in parallel*)

Key Keywords and Concepts

Keyword	Meaning
`async def`	Defines an **asynchronous function**
`await`	Tells Python to **pause here** and run something else
`asyncio.run()`	Runs the top-level coroutine
`asyncio.sleep(n)`	Non-blocking sleep (like `time.sleep` but async)
`asyncio.gather()`	Run multiple async tasks together

Real-World Example: Simulate API Calls

```
import asyncio

async def fetch_data(site):
    print(f"Fetching from {site}")
    await asyncio.sleep(1)  # simulate network delay
    print(f"Done fetching {site}")

async def main():
    sites = ['Google', 'YouTube', 'Facebook']
    tasks = [fetch_data(site) for site in sites]
    await asyncio.gather(*tasks)

asyncio.run(main())
```

Output:

```
Fetching from Google
Fetching from YouTube
Fetching from Facebook
Done fetching Google
Done fetching YouTube
Done fetching Facebook
```

Runs **in parallel**, total time ≈ 1 second.

Async vs Threading

Feature	Asyncio	Threading
Use case	I/O-bound	CPU or I/O-bound
Memory	Lightweight	Heavier
Speed	Faster switching	Slower context switching
Complexity	Cleaner, scalable	Needs locks and care

When to Use Asyncio?

- Calling APIs or doing network operations
- Waiting for user input
- Reading/writing files (non-blocking libs like aiofiles)
- Building fast web apps (like FastAPI)